Here's the **Final Project Report** for your DocSpot: Seamless Appointment Booking for Health based on the details you provided:

🛫 FINAL PROJECT REPORT DocSpot: Seamless Appointment Booking for Health

1. PROJECT OVERVIEW

Field	Detail
Project Name	DocSpot: Seamless Appointment Booking for Health
Team Members	Mandakapu Mamatha, Mamilla Jaswanth Kumar,Banda Vijay Kumar ,Diviti Sowmya ,R Jayasree
Start Date	20th May 2024
End Date	30th June 2024
Mentor/Guide	SmartInternz
Team ID	LTVIP2025TMID56112
GitHub Repo	https://github.com/Jash-1th/Health_Appointment_System
Team Leader	Mandakapu Mamatha
Email	mamathamandakapu@gmail.com

2. OBJECTIVE

To develop a responsive and functional Seamless Appointment Booking for Health using the **MERN Stack** that allows customers to search and book doctor appointment, while operators manage doctor and schedule data, and admins approve/decline new users.

3. ROLES AND RESPONSIBILITIES

Role	Capabilities
Customer	Register, log in, search doctors, book appointments

Role	Capabilities
Operator	Add, edit, delete doctor profiles and schedules ; view appointments
Admin	Approve/decline new customer/operator registrations; oversee system

4. FUNCTIONAL REQUIREMENTS

Appointment Search: Doctor speciality, location, available date

Appointment Filters: By consultation fee, hospital/clinic

Booking Page: Minimal implementation (optional)

Authentication: Secure login for all user roles

Admin Panel: Approve/reject users, manage doctor

Responsive UI: Clean and mobile-friendly interface

5. TECHNOLOGY STACK

Layer	Technology
Frontend	React JS, HTML, CSS, JavaScript
Backend	Node.js, Express.js
Database	MongoDB (via Mongoose)
Authentication	JWT (JSON Web Tokens)
Deployment	GitHub, optional: Render/Netlify/Vercel/MongoDB Atlas

6. IMPLEMENTATION PLAN (SPRINT-WISE)

Sprint	Duration	Key Tasks
1	Week 1	Requirement gathering, planning, mockup UI
2	Week 2	React setup, login/register, doctor search page
3	Week 3	Express API for authentication and appointment bookings
4	Week 4	MongoDB data connection and model creation

Sprint	Duration	Key Tasks
5	Week 5	Admin dashboard and operator CRUD for appointments
6	Week 6	Testing, debugging, documentation, and deployment

7. ER DIAGRAM (HIGH-LEVEL OVERVIEW)

```
Users
│
├── userId (PK)
├── name
├── email
├── password
├── role (customer/operator/admin)

Flights
│
├── doctorId (PK)
├── name
├── specialization
├── hospital
├── consultation fee
├── Available slots

Appointments
│
├── AppointmentId (PK)
├── userId (FK)
├── DoctorId (FK)
├── AppointmentDate
├── status
```

8. DATA FLOW DIAGRAM (DFD – LEVEL 1 OVERVIEW)

```
[Customer] → [Login/Register] → [Doctor Search] →
[Appointment Booking]
```

```
                                    ↓
                              [Backend API]
                                    ↓
                              [MongoDB DB]

[Operator] → [Login] → [Add/Edit/Delete Flights]
                                ↓
                            [Backend API]

[Admin] → [Approve/Decline Users] → [Manage Operators/
Customers]
```

9. USER STORIES

As a customer, I want to search for doctor availability using origin, hospital, and date so I can find the best options.

As an operator, I want to manage doctor appointments(add, edit, delete) so that I can keep appointments data up to date.

As an admin, I want to approve or decline new users to control access to the system.

10. TEST PLAN

Test Case	Expected Outcome	Result
Invalid login	Show error message	✅
Successful customer registration	Redirect to login	✅
Booking with invalid inputs	Block submission	✅
Admin route access by customer	Denied with 403 error	✅
Operator creates valid flight	Appointment visible to customers	✅
Duplicate booking	Prevent duplicate record	✅

11. DEBUGGING & VALIDATION

Postman used for backend API testing.

React Form Validation to block invalid inputs.

Error Logging with try-catch and console tracing.

Manual testing of:

Appointments

Route protection

Doctor filters and CRUD

12. DEPLOYMENT (SUGGESTED STEPS)

Push frontend to GitHub.

Deploy frontend via Vercel/Netlify.

Push backend code to GitHub.

Deploy backend via Render or Railway.

Use MongoDB Atlas for cloud DB access.

Add environment variables:

JWT_SECRET

DB_URL

Final integration testing post-deployment.

13. CONCLUSION

The project DocSpot: Seamless Appointment Booking for Health is a successful full-stack MERN application developed for the SmartInternz internship. It meets all functionality goals:

Efficient **customer experience**

Clear **operator management**

Secure **admin control**

The app combines real-world usability with clean architecture, demonstrating the team’s ability to execute a full lifecycle product from planning to deployment.